

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 4**



ViewModel and Debugging

Oleh:

Muhammad Erza Raihan NIM. 2310817210027

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2025**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 4

Laporan Praktikum Pemrograman Mobile Modul 4: ViewModel and Debugging ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Erza Raihan
NIM : 2310817210027

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Raka Azwar
NIM. 2210817210012

Eka Setya Wijaya S.T, M.Kom.
NIP. 198205082008011010

DAFTAR ISI

DAFTAR ISI.....	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL.....	5
SOAL 1.....	6
A. Source Code	7
B. Output Program.....	26
C. Pembahasan.....	28
SOAL 2.....	36
TAUTAN GIT	37

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Test Debugging XML	26
Gambar 2. Screenshot Hasil Test Debugging Compose.....	27

DAFTAR TABEL

Tabel 1. Source Code MainActivity XML Soal.....	8
Tabel 2. Source Code DetailActivity XML Soal 1.....	10
Tabel 3. Source Code Game XML Soal 1.....	11
Tabel 4. Source Code GameAdapter XML Soal 1	12
Tabel 5. Source Code GameViewModel XML Soal 1	14
Tabel 6. Source Code GameViewModelFactory XML Soal 1	14
Tabel 7. Source Code activity_detail XML Soal 1.....	16
Tabel 8. Source Code activity_main XML Soal 1.....	17
Tabel 9. Source Code item_game XML Soal 1.....	18
Tabel 10. Source Code MainActivity Compose Soal 1	19
Tabel 11. Source Code DetailScreen Compose Soal 1	21
Tabel 12. Source Code ListScreen Compose Soal 1	24
Tabel 13. Source Code Game Compose Soal 1.....	25
Tabel 14. Source Code GameViewModel Compose Soal 1	25

SOAL 1

Soal Praktikum:

Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
- b. Gunakan ViewModelFactory untuk membuat parameter dengan tipe data String di dalam ViewModel
- c. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment
- d. Install dan gunakan library Timber untuk logging event berikut:
 - a. Log saat data item masuk ke dalam list
 - b. Log saat tombol Detail dan tombol Explicit Intent ditekan
 - c. Log data dari list yang dipilih ketika berpindah ke halaman Detail
- e. Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi.

Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out

A. Source Code

MainActivity.kt XML

```
1 package com.example.modul3xml
2
3
4 import android.content.Intent
5 import android.net.Uri
6 import android.os.Bundle
7 import androidx.activity.viewModels
8 import androidx.appcompat.app.AppCompatActivity
9 import androidx.lifecycle.LifecycleScope
10 import androidx.recyclerview.widget.LinearLayoutManager
11 import androidx.recyclerview.widget.RecyclerView
12 import com.example.modul3xml.adapter.GameAdapter
13 import com.example.modul3xml.viewmodel.GameViewModel
14 import com.example.modul3xml.viewmodel.GameViewModelFactory
15 import kotlinx.coroutines.flow.collectLatest
16 import kotlinx.coroutines.launch
17 import timber.log.Timber
18
19
20
21
22
23 class MainActivity : AppCompatActivity() {
24
25     private lateinit var recyclerView: RecyclerView
26     private lateinit var gameAdapter: GameAdapter
27
28
29     private val viewModel: GameViewModel by viewModels {
30         GameViewModelFactory("UsernameContoh")
31     }
32
33
34     override fun onCreate(savedInstanceState: Bundle?) {
35         super.onCreate(savedInstanceState)
36         setContentView(R.layout.activity_main)
37
38
39         recyclerView = findViewById(R.id.recyclerView)
40
41
42         gameAdapter = GameAdapter(
43             context = this,
44             gameList = emptyList(),
45             onDetailClick = { id ->
46                 viewModel.onGameSelected(id)
47             },
48             onLinkClick = { url ->
49                 viewModel.onLinkClicked(url)
50                 val browserIntent = Intent(Intent.ACTION_VIEW, Uri.parse(url))
51                 startActivity(browserIntent)
52             }
53         )
54
55
56     )
57
58
59     recyclerView.apply {
60         layoutManager = LinearLayoutManager(this@MainActivity)
61     }
```

```

62         adapter = gameAdapter
63     }
64
65     lifecycleScope.launch {
66         viewModel.games.collectLatest { gameList ->
67             gameAdapter = GameAdapter(
68                 context = this@MainActivity,
69                 gameList = gameList,
70                 onDetailClick = { id ->
71                     viewModel.onGameSelected(id)
72                 },
73                 onLinkClick = { url ->
74                     viewModel.onLinkClicked(url)
75                     val browserIntent = Intent(Intent.ACTION_VIEW,
76                         Uri.parse(url))
77                     startActivity(browserIntent)
78                 }
79             )
80             recyclerView.adapter = gameAdapter
81         }
82     }
83
84     lifecycleScope.launch {
85         viewModel.selectedGameId.collectLatest { id ->
86             if (id != null) {
87                 Timber.d("Navigating to DetailActivity for game ID: $id")
88                 val intent = Intent(this@MainActivity,
89                     DetailActivity::class.java)
90                 intent.putExtra(DetailActivity.EXTRA_GAME_ID, id)
91                 startActivity(intent)
92
93                 viewModel.clearSelection()
94             }
95         }
96     }
97 }
98
99
100
101
102
103
104
105
106
107
108
109

```

Tabel 1. Source Code MainActivity XML Soal

DetailActivity.kt XML

```
1 package com.example.modul3xml
2
3
4 import android.os.Bundle
5 import android.widget.Button
6 import android.widget.ImageView
7 import android.widget.TextView
8 import androidx.appcompat.app.AppCompatActivity
9 import com.example.modul3xml.model.gameList
10 import timber.log.Timber
11
12
13
14 class DetailActivity : AppCompatActivity() {
15
16     private lateinit var detailTitle: TextView
17     private lateinit var detailYear: TextView
18     private lateinit var detailImage: ImageView
19     private lateinit var detailPlot: TextView
20     private lateinit var btnBack: Button
21
22
23
24     companion object {
25         const val EXTRA_GAME_ID = "GAME_ID"
26     }
27
28
29
30     override fun onCreate(savedInstanceState: Bundle?) {
31         super.onCreate(savedInstanceState)
32         setContentView(R.layout.activity_detail)
33
34
35         detailTitle = findViewById(R.id.detail_title)
36         detailYear = findViewById(R.id.detail_year)
37         detailImage = findViewById(R.id.detail_image)
38         detailPlot = findViewById(R.id.detail_plot)
39         btnBack = findViewById(R.id.btn_back)
40
41
42         val gameId = intent.getIntExtra(EXTRA_GAME_ID, -1)
43
44
45         val game = gameList.find { it.id == gameId }
46
47         if (game != null) {
48             Timber.d("Opening detail for game id=$gameId,
49 title=${getString(game.titleRes)}")
50
51
52             detailTitle.text = getString(game.titleRes)
53             detailYear.text = game.year
54             detailImage.setImageResource(game.imageRes)
55             detailPlot.text = getString(game.plotRes)
56         } else {
57             Timber.w("Game with id=$gameId not found!")
58             detailTitle.text = getString(R.string.game_not_found)
59             detailYear.text = ""
60             detailPlot.text = ""
61
62
63     }
```

64	
65	btnBack.setOnClickListener {
66	Timber.d("Back button pressed in DetailActivity for game
67	id=\$gameId")
68	finish()
69	}
70	}
71	}
72	
73	
74	

Tabel 2. Source Code DetailActivity XML Soal 1

Game.kt XML

1	package com.example.modul3xml.model
2	
3	
4	import com.example.modul3xml.R
5	
6	data class Game(
7	val id: Int,
8	val titleRes: Int,
9	val year: String,
10	val plotRes: Int,
11	val imageRes: Int,
12	val url: String
13	)
14	
15	
16	
17	
18	val gameList = listOf(
19	Game(
20	id = 1,
21	titleRes = R.string.name_helldivers2,
22	plotRes = R.string.desc_helldivers2,
23	imageRes = R.drawable.helldivers2,
24	url = "https://store.steampowered.com/app/553850/HELLDIVERS_2/",
25	year = "2024"
26	),
27	Game(
28	id = 2,
29	titleRes = R.string.name_p3reload,
30	plotRes = R.string.desc_p3reload,
31	imageRes = R.drawable.p3reload,
32	url =
33	"https://store.steampowered.com/app/2161700/Persona_3_Reload/",
34	year = "2024"
35	),
36	Game(
37	id = 3,
38	titleRes = R.string.name_metaphor,
39	plotRes = R.string.desc_metaphor,
40	
41	
42	
43	
44	
45	
46	

47	imageRes = R.drawable.metaphor,
48	url = "https://atlus.com/metaphor/",
49	year = "2024"
50	),
51	Game(
52	id = 4,
53	titleRes = R.string.name_berseria,
54	plotRes = R.string.desc_berseria,
55	imageRes = R.drawable.berseria,
56	url =
57	"https://store.steampowered.com/app/429660/Tales_of_Berseria/",
58	year = "2016"
59	),
60	Game(
61	id = 5,
62	titleRes = R.string.name_p5royal,
63	plotRes = R.string.desc_p5royal,
64	imageRes = R.drawable.p5royal,
65	url = "https://store.steampowered.com/app/1687950/Persona_5_Royal/",
66	year = "2019"
67	)
68	)
69	)
70	)

Tabel 3. Source Code Game XML Soal 1

GameAdapter.kt XML

1	package com.example.modul3xml.adapter
2	
3	
4	import android.content.Context
5	import android.view.LayoutInflater
6	import android.view.View
7	import android.view.ViewGroup
8	import android.widget.Button
9	import android.widget.ImageView
10	import android.widget.TextView
11	import androidx.recyclerview.widget.RecyclerView
12	import com.example.modul3xml.R
13	import com.example.modul3xml.model.Game
14	import timber.log.Timber
15	
16	class GameAdapter(
17	private val context: Context,
18	private val gameList: List<Game>,
19	private val onDetailClick: (Int) -> Unit,
20	private val onLinkClick: (String) -> Unit
21	) : RecyclerView.Adapter<GameAdapter.GameViewHolder>() {
22	
23	inner class GameViewHolder(view: View) : RecyclerView.ViewHolder(view) {
24	val imageGame: ImageView = view.findViewById(R.id.image_game)
25	
26	
27	
28	
29	
30	

31	val textTitle: TextView = view.findViewById(R.id.text_title)
32	val textYear: TextView = view.findViewById(R.id.text_year)
33	val buttonDetail: Button = view.findViewById(R.id.btn_detail)
34	val buttonPage: Button = view.findViewById(R.id.btn_link)
35	
36	}
37	
38	
39	override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
40	GameViewHolder {
41	val view = LayoutInflater.from(context).inflate(R.layout.item_game,
42	parent, false)
43	return GameViewHolder(view)
44	}
45	
46	
47	
48	override fun onBindViewHolder(holder: GameViewHolder, position: Int) {
49	val game = gameList[position]
50	
51	holder.imageGame.setImageResource(game.imageRes)
52	holder.textTitle.text = context.getString(game.titleRes)
53	holder.textYear.text = game.year.toString()
54	
55	
56	holder.buttonDetail.setOnClickListener {
57	Timber.d("Detail button clicked for gameId: \${game.id}")
58	onDetailClick(game.id)
59	}
60	
61	
62	holder.buttonPage.setOnClickListener {
63	Timber.d("Explicit Intent button clicked with URL: \${game.url}")
64	onLinkClick(game.url)
65	}
66	
67	
68	Timber.d("Game added to list: \${game.titleRes}")
69	}
70	
71	
72	override fun getItemCount(): Int = gameList.size
73	}

Tabel 4. Source Code GameAdapter XML Soal 1

GameViewModel.kt XML

```
1 package com.example.modul3xml.adapter
2
3
4 import android.content.Context
5 import android.view.LayoutInflater
6 import android.view.View
7 import android.view.ViewGroup
8 import android.widget.Button
9 import android.widget.ImageView
10 import android.widget.TextView
11 import androidx.recyclerview.widget.RecyclerView
12 import com.example.modul3xml.R
13 import com.example.modul3xml.model.Game
14 import timber.log.Timber
15
16
17
18
19 class GameAdapter(
20     private val context: Context,
21     private val gameList: List<Game>,
22     private val onDetailClick: (Int) -> Unit,
23     private val onLinkClick: (String) -> Unit
24 ) : RecyclerView.Adapter<GameAdapter.GameViewHolder>() {
25
26     inner class GameViewHolder(view: View) : RecyclerView.ViewHolder(view) {
27         val imageGame: ImageView = view.findViewById(R.id.image_game)
28         val textTitle: TextView = view.findViewById(R.id.text_title)
29         val textYear: TextView = view.findViewById(R.id.text_year)
30         val buttonDetail: Button = view.findViewById(R.id.btn_detail)
31         val buttonPage: Button = view.findViewById(R.id.btn_link)
32     }
33
34     override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
35     GameViewHolder {
36         val view = LayoutInflater.from(context).inflate(R.layout.item_game,
37         parent, false)
38         return GameViewHolder(view)
39     }
40
41     override fun onBindViewHolder(holder: GameViewHolder, position: Int) {
42         val game = gameList[position]
43
44         holder.imageGame.setImageResource(game.imageRes)
45         holder.textTitle.text = context.getString(game.titleRes)
46         holder.textYear.text = game.year.toString()
47
48         holder.buttonDetail.setOnClickListener {
49             Timber.d("Detail button clicked for gameId: ${game.id}")
50             onDetailClick(game.id)
51         }
52
53         holder.buttonPage.setOnClickListener {
54             Timber.d("Explicit Intent button clicked with URL: ${game.url}")
55         }
56     }
57 }
```

64	onLinkClick(game.url)
65	}
66	
67	Timber.d("Game added to list: \${game.titleRes}")
68	}
69	
70	
71	override fun getItemCount(): Int = gameList.size
72	}
73	
74	

Tabel 5. Source Code GameViewModel XML Soal 1

GameViewModelFactory.kt XML

1	package com.example.modul3xml.viewmodel
2	
3	
4	import androidx.lifecycle.ViewModel
5	import androidx.lifecycle.ViewModelProvider
6	
7	
8	class GameViewModelFactory(private val username: String) :
9	ViewModelProvider.Factory {
10	override fun <T : ViewModel> create(modelClass: Class<T>): T {
11	if (modelClass.isAssignableFrom(GameViewModel::class.java)) {
12	return GameViewModel(username) as T
13	}
14	throw IllegalArgumentException("Unknown ViewModel class")
15	}
16	
17	}
18	

Tabel 6. Source Code GameViewModelFactory XML Soal 1

activity_detail.xml

```
1  <?xml version="1.0" encoding="utf-8"?>
2
3  <ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
4      android:layout_width="match_parent"
5      android:layout_height="match_parent"
6      android:padding="16dp"
7      android:background="#D2D2D2">
8
9
10     <LinearLayout
11         android:layout_width="match_parent"
12         android:layout_height="wrap_content"
13         android:orientation="vertical"
14         android:gravity="center_horizontal">
15
16
17         <TextView
18             android:id="@+id/detail_title"
19             android:layout_width="wrap_content"
20             android:layout_height="wrap_content"
21             android:text="GAME TITLE"
22             android:textSize="24sp"
23             android:textStyle="bold"
24             android:textAlignment="center"
25             android:textColor="#313131"
26             android:paddingBottom="8dp" />
27
28
29         <TextView
30             android:id="@+id/detail_year"
31             android:layout_width="wrap_content"
32             android:layout_height="wrap_content"
33             android:text="2024"
34             android:textSize="16sp"
35             android:textColor="#313131"
36             android:paddingBottom="16dp" />
37
38
39         <ImageView
40             android:id="@+id/detail_image"
41             android:layout_width="match_parent"
42             android:layout_height="250dp"
43             android:scaleType="centerCrop"
44             android:background="#FFFFFF" />
45
46
47         <TextView
48             android:id="@+id/detail_plot_label"
49             android:layout_width="wrap_content"
50             android:layout_height="wrap_content"
51             android:text="Plot:"
52             android:textStyle="bold"
53             android:textSize="18sp"
54             android:textColor="#313131"
55             android:paddingTop="16dp" />
56
57
58
59
60
61
62
63
```

64	<TextView
65	android:id="@+id/detail_plot"
66	android:layout_width="match_parent"
67	android:layout_height="wrap_content"
68	android:text="..."
69	android:textColor="#313131" />
70	
71	
72	<Button
73	android:id="@+id/btn_back"
74	android:layout_width="wrap_content"
75	android:layout_height="wrap_content"
76	android:text="Back to List"
77	android:textColor="#313131"
78	android:backgroundTint="#FFA31A"
79	android:layout_marginTop="24dp" />
80	
81	</LinearLayout>
82	</ScrollView>
83	
84	

Tabel 7. Source Code activity_detail XML Soal 1

activity_main.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3	android:id="@+id/mainLayout"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent"
6	android:orientation="vertical"
7	android:padding="16dp"
8	android:background="#CCC8C8">
9	
10	
11	<TextView
12	android:id="@+id/textTitle"
13	android:layout_width="wrap_content"
14	android:layout_height="wrap_content"
15	android:text="Top 5 Game Pick by Eza"
16	android:textSize="24sp"
17	android:textStyle="bold"
18	android:textColor="#313131"
19	android:layout_marginBottom="8dp" />
20	
21	
22	<androidx.recyclerview.widget.RecyclerView
23	android:id="@+id/recyclerView"
24	android:layout_width="match_parent"
25	android:layout_height="wrap_content"
26	android:clipToPadding="false"
27	android:paddingBottom="16dp"
28	
29	
30	
31	
32	

33	android:scrollbars="vertical"/>
34	</LinearLayout>
35	

Tabel 8. Source Code activity_main XML Soal 1

Item_game.xml

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.cardview.widget.CardView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:card_view="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="wrap_content"
7	android:layout_marginBottom="12dp"
8	card_view:cardCornerRadius="12dp"
9	card_view:cardElevation="6dp"
10	android:background="#FFFFFF">
11	
12	<LinearLayout
13	android:layout_width="match_parent"
14	android:layout_height="wrap_content"
15	android:orientation="vertical"
16	android:padding="16dp">
17	
18	<ImageView
19	android:id="@+id/image_game"
20	android:layout_width="match_parent"
21	android:layout_height="180dp"
22	android:scaleType="centerCrop"
23	android:contentDescription="@string/game_image_desc" />
24	
25	<TextView
26	android:id="@+id/text_title"
27	android:layout_width="wrap_content"
28	android:layout_height="wrap_content"
29	android:text="Game Title"
30	android:textSize="18sp"
31	android:textStyle="bold"
32	android:textColor="#313131"
33	android:layout_marginTop="8dp" />
34	
35	<TextView
36	android:id="@+id/text_year"
37	android:layout_width="wrap_content"
38	android:layout_height="wrap_content"
39	android:text="2024"
40	android:textColor="#666666"
41	android:layout_marginBottom="8dp" />
42	
43	
44	
45	
46	
47	
48	
49	
50	

51	
52	<LinearLayout
53	android:layout_width="match_parent"
54	android:layout_height="wrap_content"
55	android:orientation="horizontal">
56	
57	<Button
58	android:id="@+id/btn_detail"
59	android:layout_width="0dp"
60	android:layout_weight="1"
61	android:layout_height="wrap_content"
62	android:text="Detail"
63	android:backgroundTint="#FFA31A"
64	android:textColor="#FFFFFF" />
65	
66	<Button
67	android:id="@+id/btn_link"
68	android:layout_width="0dp"
69	android:layout_weight="1"
70	android:layout_height="wrap_content"
71	android:text="Open Link"
72	android:layout_marginStart="8dp"
73	android:backgroundTint="#FFA31A"
74	android:textColor="#FFFFFF" />
75	
76	</LinearLayout>
77	</LinearLayout>
78	
79	</androidx.cardview.widget.CardView>
80	

Tabel 9. Source Code item_game XML Soal 1

MainActivity.kt Compose

1	package com.example.modul3compose
2	import android.content.Intent
3	import android.net.Uri
4	import android.os.Bundle
5	import androidx.activity.ComponentActivity
6	import androidx.activity.compose.setContent
7	import androidx.activity.viewModels
8	import androidx.compose.runtime.Composable
9	import androidx.compose.runtime.collectAsState
10	import androidx.compose.runtime.getValue
11	import androidx.lifecycle.lifecycleScope
12	import com.example.modul3compose.ui.theme.MODUL3COMPOSETheme
13	import com.example.modul3compose.viewmodel.GameViewModel
14	import com.example.modul3compose.viewmodel.GameViewModelFactory
15	import kotlinx.coroutines.flow.launchIn
16	import kotlinx.coroutines.flow.onEach
17	import timber.log.Timber
18	class MainActivity : ComponentActivity() {

```

19     private val viewModel: GameViewModel by viewModels {
20         GameViewModelFactory("Eza")
21     }
22
23     override fun onCreate(savedInstanceState: Bundle?) {
24         super.onCreate(savedInstanceState)
25
26         viewModel.navigateToUrl
27             .onEach { url ->
28                 url?.let { openBrowser(it) }
29             }
30             .launchIn(lifecycleScope)
31         setContent {
32             MODUL3COMPOSETheme {
33                 GameApp(viewModel = viewModel)
34             }
35         }
36     }
37     private fun openBrowser(url: String) {
38         Timber.d("Opening browser with URL: $url")
39         val intent = Intent(Intent.ACTION_VIEW, Uri.parse(url))
40         startActivity(intent)
41         viewModel.onUrlNavigated()
42     }
43 }
44 @Composable
45 fun GameApp(viewModel: GameViewModel) {
46     val selectedGameId by viewModel.selectedGameId.collectAsState()
47     val gameList by viewModel.games.collectAsState()
48
49     if (selectedGameId == null) {
50         ListScreen(
51             gameList = gameList,
52             onDetailClick = { viewModel.onGameSelected(it) },
53             onLinkClick = { viewModel.onLinkClicked(it) }
54         )
55     } else {
56         selectedGameId?.let { id ->
57             DetailScreen(
58                 game = gameList.first { it.id == id },
59                 onBack = { viewModel.clearSelection() }
60             )
61         }
62     }
63 }
64

```

Tabel 10. Source Code MainActivity Compose Soal 1

DetailScreen.kt Compose

```

1     package com.example.modul3compose
2
3
4     import androidx.compose.foundation.Image
5     import androidx.compose.foundation.layout.Column
6     import androidx.compose.foundation.layout.fillMaxSize
7     import androidx.compose.foundation.layout.fillMaxWidth
8     import androidx.compose.foundation.layout.height
9     import androidx.compose.foundation.layout.padding
10    import androidx.compose.foundation.rememberScrollState
11    import androidx.compose.foundation.verticalScroll
12    import androidx.compose.material.icons.Icons
13    import androidx.compose.material.icons.filled.ArrowBack
14    import androidx.compose.material3.Button
15    import androidx.compose.material3.ButtonDefaults
16    import androidx.compose.material3.ExperimentalMaterial3Api
17    import androidx.compose.material3.Icon
18

```

```

17 import androidx.compose.material3.IconButton
18 import androidx.compose.material3.MaterialTheme
19 import androidx.compose.material3.Scaffold
20 import androidx.compose.material3.Text
21 import androidx.compose.material3.TopAppBar
22 import androidx.compose.material3.TopAppBarDefaults
23 import androidx.compose.runtime.Composable
24 import androidx.compose.ui.Alignment
25 import androidx.compose.ui.Modifier
26 import androidx.compose.ui.draw.clip
27 import androidx.compose.ui.layout.ContentScale
28 import androidx.compose.ui.res.painterResource
29 import androidx.compose.ui.res.stringResource
30 import androidx.compose.ui.text.font.FontWeight
31 import androidx.compose.ui.text.style.TextAlign
32 import androidx.compose.ui.unit.dp
33 import androidx.compose.ui.unit.sp
34 import com.example.modul3compose.model.Game
35 import com.example.modul3compose.ui.theme.Black1
36 import com.example.modul3compose.ui.theme.softgrey1
37 import com.example.modul3compose.ui.theme.yellow1
38 import timber.log.Timber
39
40 @OptIn(ExperimentalMaterial3Api::class)
41 @Composable
42 fun DetailScreen(
43     game: Game,
44     onBackPressed: () -> Unit
45 ) {
46     Timber.d("Navigated to DetailScreen with game: ${game.titleRes}, year:
47     ${game.year}")
48
49     Scaffold(
50         topBar = {
51             TopAppBar(
52                 title = { Text(stringResource(id = game.titleRes),
53                     color = Black1 ) },
54                 navigationIcon = {
55                     IconButton(onClick = onBackPressed) {
56                         Icon(
57                             imageVector = Icons.Filled.ArrowBack,
58                             contentDescription = "Back",
59                             tint = Black1
60                         )
61                     }
62                 },
63                 colors = TopAppBarDefaults.smallTopAppBarColors(
64                     containerColor = yellow1,
65                     titleContentColor = MaterialTheme.colorScheme.onPrimary,
66                 )
67             )
68         },
69         containerColor = softgrey1
70     ) { innerPadding ->
71         Column(
72             modifier = Modifier
73                 .padding(innerPadding)
74                 .verticalScroll(rememberScrollState())
75                 .fillMaxSize(),
76             horizontalAlignment = Alignment.CenterHorizontally
77         ) {
78             Text(
79                 text = stringResource(id =
80                 game.titleRes).uppercase().replace(" ", "\n"),
81                 style = MaterialTheme.typography.headlineLarge.copy(
82                     fontSize = 36.sp,

```

84	fontWeight = FontWeight.Bold,
85	textAlign = TextAlign.Center,
86	color = <i>Black1</i>
87	),
88	modifier = Modifier. <i>padding</i> (vertical = 16.dp)
89	)
90	Text(
91	text = game.year,
92	style = MaterialTheme.typography.titleMedium,
93	color = <i>Black1</i> ,
94	modifier = Modifier. <i>padding</i> (bottom = 16.dp)
95	)
96	
97	Image(
98	painter = painterResource(id = game.imageRes),
99	contentDescription = stringResource(id = game.titleRes),
100	modifier = Modifier
101	. <i>fillMaxWidth</i> ()
102	. <i>height</i> (250.dp)
103	. <i>clip</i> (MaterialTheme.shapes.medium),
104	contentScale = ContentScale.Crop
105	)
106	
107	Column(
108	modifier = Modifier
109	. <i>fillMaxWidth</i> ()
110	. <i>padding</i> (16.dp)
111	) {
112	Text(
113	text = "Plot:",
114	style = MaterialTheme.typography.titleLarge,
115	fontWeight = FontWeight.Bold,
116	modifier = Modifier. <i>padding</i> (bottom = 8.dp)
117	)
118	Text(
119	text = stringResource(id = game.plotRes),
120	style = MaterialTheme.typography.bodyLarge,
121	modifier = Modifier. <i>padding</i> (bottom = 16.dp)
122	)
123	}
124	
125	Button(
126	onClick = onBackPressed,
127	modifier = Modifier
128	. <i>padding</i> (16.dp)
129	. <i>fillMaxWidth</i> (0.8f),
130	shape = MaterialTheme.shapes.medium,
131	colors = ButtonDefaults.buttonColors(
132	containerColor = <i>yellow1</i>
133	)) {
134	Text("Back to List")
135	}
136	}
137	}
138	}

Tabel 11. Source Code DetailScreen Compose Soal 1

ListScreen.kt Compose

```
1 package com.example.modul3compose
2
3 import androidx.compose.foundation.Image
4 import androidx.compose.foundation.layout.Arrangement
5 import androidx.compose.foundation.layout.Column
6 import androidx.compose.foundation.layout.Row
7 import androidx.compose.foundation.layout.Spacer
8 import androidx.compose.foundation.layout.fillMaxWidth
9 import androidx.compose.foundation.layout.height
10 import androidx.compose.foundation.layout.padding
11 import androidx.compose.foundation.lazy.LazyColumn
12 import androidx.compose.foundation.lazy.items
13 import androidx.compose.foundation.shape.RoundedCornerShape
14 import androidx.compose.material3.Button
15 import androidx.compose.material3.ButtonDefaults
16 import androidx.compose.material3.Card
17 import androidx.compose.material3.CardDefaults
18 import androidx.compose.material3.ExperimentalMaterial3Api
19 import androidx.compose.material3.MaterialTheme
20 import androidx.compose.material3.Scaffold
21 import androidx.compose.material3.Text
22 import androidx.compose.material3.TopAppBar
23 import androidx.compose.material3.TopAppBarDefaults
24 import androidx.compose.runtime.Composable
25 import androidx.compose.ui.Modifier
26 import androidx.compose.ui.draw.clip
27 import androidx.compose.ui.layout.ContentScale
28 import androidx.compose.ui.res.painterResource
29 import androidx.compose.ui.text.font.FontWeight
30 import androidx.compose.ui.unit.dp
31 import com.example.modul3compose.model.Game
32 import com.example.modul3compose.ui.theme.Black1
33 import com.example.modul3compose.ui.theme.softgrey1
34 import com.example.modul3compose.ui.theme.yellow1
35
36 @OptIn(ExperimentalMaterial3Api::class)
37 @Composable
38 fun ListScreen(
39     gameList: List<Game>,
40     onDetailClick: (Int) -> Unit,
41     onLinkClick: (String) -> Unit
42 ) {
43     Scaffold(
44         topBar = {
45             TopAppBar(
46                 title = { Text("Top 5 Game pick by Eza") },
47                 colors = TopAppBarDefaults.smallTopAppBarColors(
48                     containerColor = yellow1,
49                     titleContentColor = MaterialTheme.colorScheme.onPrimary
50                 )
51             )
52         },
53         containerColor = MaterialTheme.colorScheme.background
54     ) { innerPadding ->
55         LazyColumn(
56             contentPadding = innerPadding,
57             verticalArrangement = Arrangement.spacedBy(8.dp)
58         ) {
59             items(gameList) { game ->
60                 GameItem(
61                     game = game,
62                     onDetailClick = { onDetailClick(game.id) },
63                     onLinkClick = { onLinkClick(game.Url) }
64                 )
65             }
66         }
67     }
68 }
```

```

65         }
66     }
67 }
68
69
70 @Composable
71 fun GameItem(
72     game: Game,
73     onDetailClick: () -> Unit,
74     onLinkClick: () -> Unit
75 ) {
76     Card(
77         modifier = Modifier
78             .padding(8.dp)
79             .fillMaxWidth(),
80         shape = RoundedCornerShape(16.dp),
81         elevation = CardDefaults.cardElevation(8.dp),
82         colors = CardDefaults.cardColors(
83             containerColor = softgrey1
84         )
85     ) {
86         Column(modifier = Modifier.padding(16.dp)) {
87             Image(
88                 painter = painterResource(id = game.imageRes),
89                 contentDescription = stringResource(id = game.titleRes),
90                 modifier = Modifier
91                     .fillMaxWidth()
92                     .height(180.dp)
93                     .clip(RoundedCornerShape(12.dp)),
94                 contentScale = ContentScale.Crop
95             )
96             Spacer(Modifier.height(8.dp))
97             Text(
98                 text = stringResource(id = game.titleRes),
99                 color = Black1,
100                style = MaterialTheme.typography.titleMedium,
101                fontWeight = FontWeight.Bold
102            )
103            Spacer(Modifier.height(4.dp))
104            Text(
105                text = stringResource(id = game.plotRes),
106                color = Black1,
107                style = MaterialTheme.typography.bodyMedium,
108                maxLines = 2
109            )
110            Spacer(Modifier.height(8.dp))
111            Row(
112                horizontalArrangement = Arrangement.SpaceBetween,
113                modifier = Modifier.fillMaxWidth()
114            ) {
115                Button(
116                    onClick = onLinkClick,
117                    colors = ButtonDefaults.buttonColors(
118                        containerColor = yellow1
119                    )
120                ) {
121                    Text("Game Page", color = Black1)
122                }
123                Button(
124                    onClick = onDetailClick,
125                    colors = ButtonDefaults.buttonColors(
126                        containerColor = yellow1
127                    )
128                ) {
129                    Text("Detail")
130                }
131            }
132        }
133    }
134 }

```

132	}
133	
134	

Tabel 12. Source Code ListScreen Compose Soal 1

Game.kt

1	package com.example.modul3compose.model
2	
3	import com.example.modul3compose.R
4	
5	data class Game(
6	val id: Int,
7	val titleRes: Int,
8	val year: String,
9	val plotRes: Int,
10	val imageRes: Int,
11	val Url: String
12	)
13	
14	
15	val gameList = listOf(
16	Game(
17	id = 1,
18	titleRes = R.string.name_helldivers2,
19	plotRes = R.string.desc_helldivers2,
20	imageRes = R.drawable.helldivers2,
21	Url = "https://store.steampowered.com/app/553850/HELLDIVERS_2/",
22	year = "2024"
23	),
24	Game(
25	id = 2,
26	titleRes = R.string.name_p3reload,
27	plotRes = R.string.desc_p3reload,
28	imageRes = R.drawable.persona3,
29	Url = "https://store.steampowered.com/app/2161700/Persona_3_Reload/",
30	year = "2024"
31	),
32	Game(
33	id = 3,
34	titleRes = R.string.name_metaphor,
35	plotRes = R.string.desc_metaphor,
36	imageRes = R.drawable.metaphor,
37	Url = "https://atlus.com/metaphor/",
38	year = "2024"
39	),
40	Game(
41	id = 4,
42	titleRes = R.string.name_berseria,
43	plotRes = R.string.desc_berseria,
44	imageRes = R.drawable.berseria,
45	Url = "https://store.steampowered.com/app/429660/Tales_of_Berseria/",
46	year = "2016"
47	),
48	Game(
49	id = 5,
50	titleRes = R.string.name_p5royal,
51	plotRes = R.string.desc_p5royal,
52	imageRes = R.drawable.persona5,
53	Url = "https://store.steampowered.com/app/1687950/Persona_5_Royal/",
54	year = "2019"
55	)
56	)

Tabel 13. Source Code Game Compose Soal 1

GameViewModel.kt

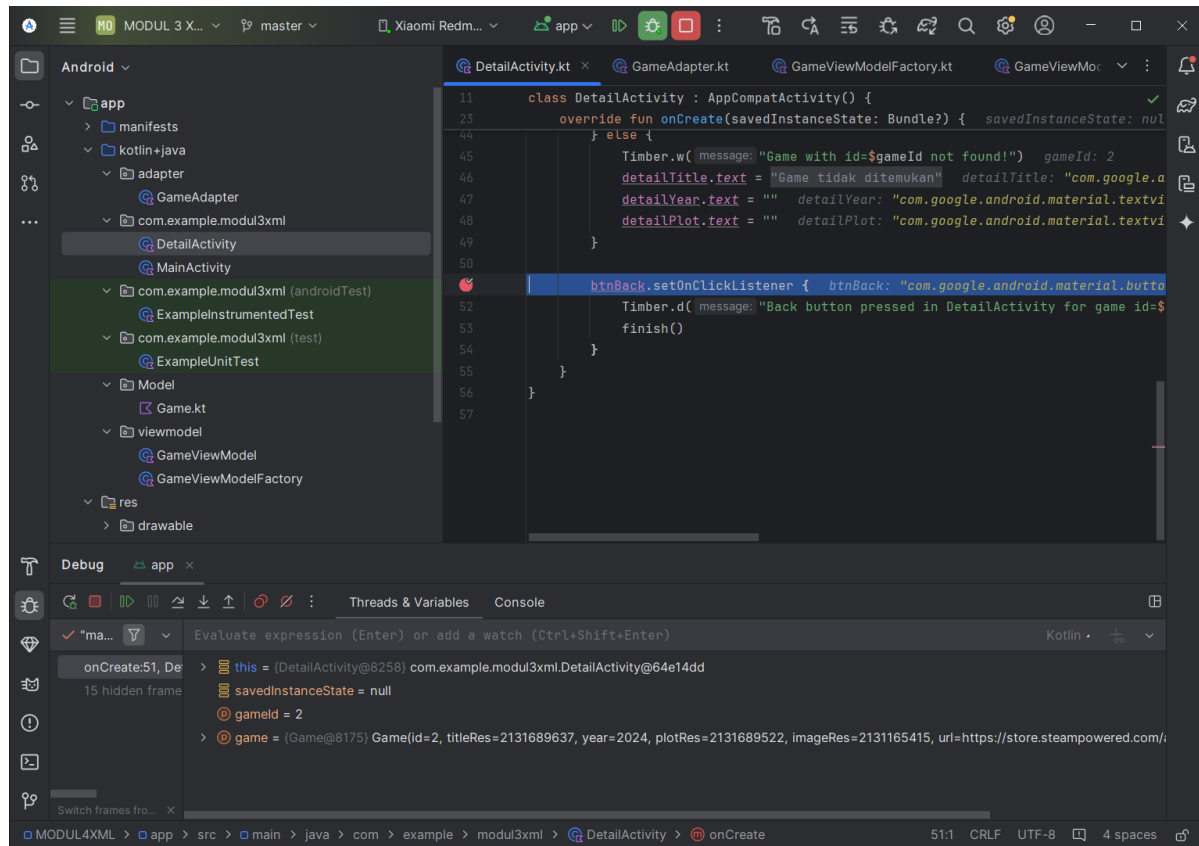
```

1 package com.example.modul3compose.viewmodel
2
3 import androidx.lifecycle.ViewModel
4 import androidx.lifecycle.ViewModelProvider
5 import com.example.modul3compose.model.Game
6 import com.example.modul3compose.model.gameList
7 import kotlinx.coroutines.flow.MutableStateFlow
8 import kotlinx.coroutines.flow.StateFlow
9 import kotlinx.coroutines.flow.asStateFlow
10 import timber.log.Timber
11
12 class GameViewModel(private val username: String) : ViewModel() {
13     private val _games = MutableStateFlow<List<Game>>(emptyList())
14     val games: StateFlow<List<Game>> = _games.asStateFlow()
15
16     private val _selectedGameId = MutableStateFlow<Int?>(null)
17     val selectedGameId: StateFlow<Int?> = _selectedGameId.asStateFlow()
18
19     private val _navigateToUrl = MutableStateFlow<String?>(null)
20     val navigateToUrl: StateFlow<String?> = _navigateToUrl.asStateFlow()
21
22     init {
23         Timber.d("User: $username - Loading games")
24         _games.value = gameList
25         Timber.d("Game list loaded: ${_games.value.size} items")
26     }
27
28     fun onGameSelected(id: Int) {
29         Timber.d("Game with ID $id selected")
30         _selectedGameId.value = id
31     }
32
33     fun clearSelection() {
34         _selectedGameId.value = null
35     }
36
37     fun onLinkClicked(url: String) {
38         Timber.d("Explicit Intent triggered for URL: $url")
39         _navigateToUrl.value = url
40     }
41
42     fun onUrlNavigated() {
43         _navigateToUrl.value = null
44     }
45 }
46
47 class GameViewModelFactory(private val username: String) :
48     ViewModelProvider.Factory {
49     override fun <T : ViewModel> create(modelClass: Class<T>): T {
50         if (modelClass.isAssignableFrom(GameViewModel::class.java)) {
51             @Suppress("UNCHECKED_CAST")
52             return GameViewModel(username) as T
53         }
54         throw IllegalArgumentException("Unknown ViewModel class")
55     }
56 }

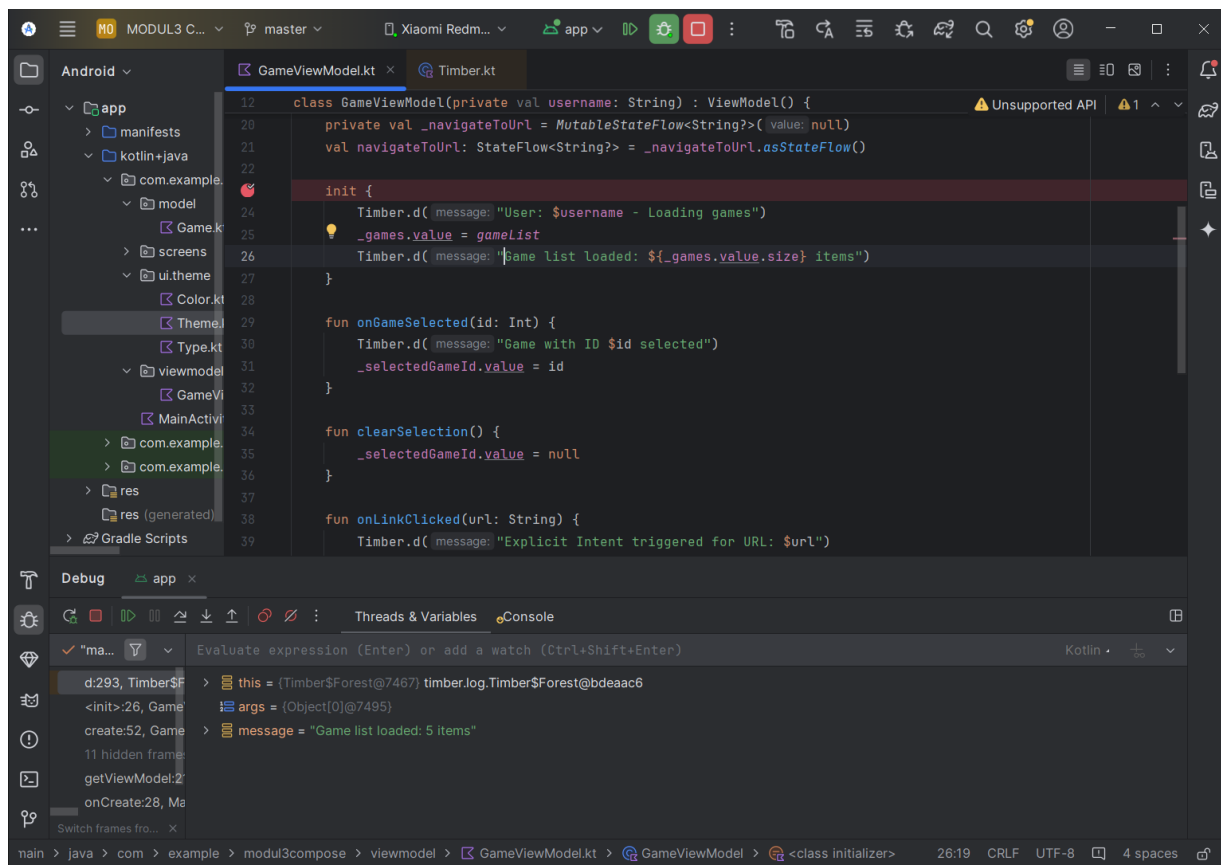
```

Tabel 14. Source Code GameViewModel Compose Soal 1

B. Output Program



Gambar 1. Screenshot Hasil Test Debugging XML



Gambar 2. Screenshot Hasil Test Debugging Compose

C. Pembahasan

MainActivity.kt XML:

MainActivity dalam aplikasi Android berbasis XML yang menggunakan arsitektur MVVM (Model-View-ViewModel) dengan dukungan StateFlow untuk manajemen state yang reaktif. Aktivitas ini menampilkan daftar game menggunakan RecyclerView dan GameAdapter, serta menghubungkannya dengan GameViewModel yang bertugas mengelola data dan logika aplikasi. Ketika pengguna mengklik tombol detail, ID game akan diteruskan ke ViewModel dan mengarahkan ke DetailActivity. Jika pengguna mengklik tautan, aplikasi akan membuka URL di browser menggunakan Intent. Penggunaan lifecycleScope.launch dengan collectLatest memastikan UI selalu merespons data terbaru dari aliran games dan selectedGameId. Setiap kali daftar game berubah, adapter diperbarui untuk menampilkan daftar baru. Timber digunakan untuk mencatat log saat navigasi dilakukan, membantu proses debugging.

Game.kt XML:

File Game.kt dalam package model ini mendefinisikan data class bernama Game yang merepresentasikan struktur data untuk setiap item game yang akan ditampilkan dalam aplikasi. Setiap objek Game memiliki beberapa properti: id, titleRes dan plotRes (ID untuk string resource judul dan deskripsi), imageRes (ID drawable untuk gambar), url (tautan ke halaman resmi game), dan year (tahun rilis).

GameAdapter:

Adapter ini menerima data berupa List<Game> serta dua lambda function (onDetailClick dan onLinkClick) yang akan dijalankan ketika tombol “Detail” atau “Link” ditekan oleh pengguna. Setiap item dalam daftar diatur melalui layout item_game.xml, yang terdiri dari ImageView untuk gambar game, TextView untuk judul dan tahun rilis, serta dua Button untuk aksi detail dan link.

Metode `onBindViewHolder` menghubungkan data game ke tampilan masing-masing item berdasarkan posisi. Saat tombol detail diklik, ID game dikirim ke callback `onDetailClick`, yang kemudian akan mengarah ke `DetailActivity`. Sementara saat tombol link ditekan, URL game dibuka di browser menggunakan `onLinkClick`. Logging menggunakan Timber (Timber.d) digunakan di berbagai titik untuk memudahkan debugging, seperti saat tombol ditekan atau game ditambahkan ke tampilan.

DetailActivity.kt XML:

`DetailActivity` ini merupakan halaman detail untuk menampilkan informasi lengkap dari sebuah game yang dipilih dari daftar di `MainActivity`. Saat Activity ini dimulai (`onCreate`), ia mengambil `gameId` yang dikirim melalui Intent dengan key `EXTRA_GAME_ID`. Kemudian, `gameList` (daftar game statis dari model) digunakan untuk mencari game dengan ID yang sesuai.

Jika game ditemukan, informasi seperti judul, tahun rilis, gambar, dan plot deskripsi ditampilkan pada UI menggunakan `TextView` dan `ImageView`. Semua teks yang ditampilkan diambil dari resource (`getString(game.titleRes)`), yang memungkinkan dukungan multibahasa. Jika ID game tidak ditemukan, aplikasi akan menampilkan pesan “game not found”. Tombol "Back" memiliki fungsi untuk menutup `DetailActivity` dan kembali ke halaman sebelumnya. Logging menggunakan Timber juga digunakan di sini untuk mencatat aktivitas penting seperti pembukaan detail game dan penekanan tombol kembali,

GameViewModel.kt XML:

`GameViewModel` adalah bagian dari arsitektur MVVM yang bertanggung jawab untuk mengelola data dan logika aplikasi secara terpisah dari UI. `ViewModel` ini menggunakan Kotlin Coroutines dengan `StateFlow` agar dapat mengelola dan membagikan state aplikasi secara reaktif. `ViewModel` ini menyimpan dua state utama:

1. `games`: berisi daftar game (`List<Game>`) yang akan ditampilkan di UI. Data ini diisi dari `gameList` saat `ViewModel` diinisialisasi melalui fungsi `loadGames()`.
2. `selectedGameId`: berisi ID game yang dipilih pengguna ketika tombol "Detail" diklik.

Fungsi `onGameSelected(id: Int)` akan memperbarui `selectedGameId`, yang kemudian diamati di `MainActivity` untuk berpindah ke `DetailActivity`. Fungsi `onLinkClicked(url: String)` hanya mencatat log saat tombol "Link" ditekan. Fungsi `clearSelection()` digunakan untuk mereset ID setelah navigasi dilakukan agar tidak terjadi navigasi ulang yang tidak disengaja. Timber digunakan untuk mencatat proses penting (seperti inisialisasi, pemilihan game, dan pemuatan data) guna mempermudah proses debugging dan pelacakan aktivitas `ViewModel`.

GameViewModelFactory.kt XML:

`GameViewModelFactory` adalah implementasi dari `ViewModelProvider.Factory` yang digunakan untuk membuat instance `GameViewModel` dengan parameter tambahan, yaitu `username`. Secara default, `ViewModel` di Android tidak mendukung konstruktor dengan parameter. Oleh karena itu, kita perlu membuat factory class ini untuk meng-inject data (seperti `username`) ke dalam `ViewModel`. Fungsi `create()` dalam factory akan dipanggil oleh sistem saat `ViewModel` pertama kali dibuat. Di dalamnya, dilakukan pengecekan apakah kelas `ViewModel` yang diminta adalah `GameViewModel`. Jika iya, maka dibuat dan dikembalikan objek `GameViewModel` menggunakan parameter `username`. Jika bukan, maka dilempar `IllegalArgumentException`.

activity_main.xml:

`activity_main.xml` digunakan oleh `MainActivity` untuk menampilkan antarmuka utama aplikasi. Layout ini menggunakan `LinearLayout` dengan orientasi vertikal agar elemen-elemen ditampilkan dari atas ke bawah. Warna latar belakang layout diatur ke abu-abu muda (`#CCC8C8`) untuk memberikan kesan lembut dan netral pada tampilan. Di bagian atas layout terdapat sebuah `TextView` dengan teks "Top 5 Game Pick by Eza", yang berfungsi sebagai judul halaman. Teks ini diatur memiliki ukuran 24sp, gaya tebal, dan warna teks gelap agar menonjol di atas latar abu-abu. Margin bawah juga ditambahkan agar terdapat jarak antara judul dan elemen di bawahnya.

Komponen utama berikutnya adalah RecyclerView, yang berfungsi untuk menampilkan daftar game secara dinamis dalam bentuk scroll vertikal. RecyclerView ini akan diisi oleh GameAdapter dalam kode Kotlin. Atribut clipToPadding diset false agar konten tetap bisa tampil di luar padding jika diperlukan, dan padding bawah ditambahkan untuk memberi ruang ekstra saat di *scroll*.

activity_detail.xml:

activity_detail.xml digunakan oleh DetailActivity untuk menampilkan informasi lengkap mengenai sebuah game. Layout utama menggunakan ScrollView, memungkinkan konten di dalamnya untuk digulir secara vertikal. Hal ini penting karena halaman detail bisa berisi banyak informasi, seperti gambar dan deskripsi panjang. Seluruh isi dari ScrollView dibungkus dalam LinearLayout yang berorientasi vertikal dan dipusatkan secara horizontal agar tampilan lebih seimbang di berbagai ukuran layar.

Elemen pertama adalah TextView dengan ID detail_title, yang menampilkan judul game dengan ukuran teks besar dan tebal untuk memberi penekanan visual. Di bawahnya, ada detail_year yang menunjukkan tahun rilis game, ditampilkan dalam ukuran lebih kecil. Selanjutnya, sebuah ImageView digunakan untuk menampilkan gambar game dalam ukuran tetap 250dp dan dengan skala centerCrop agar gambar mengisi ruang secara proporsional. Setelah itu, terdapat label Plot: yang dicetak tebal sebagai penanda bagian deskripsi, diikuti oleh TextView detail_plot yang menampilkan sinopsis atau latar belakang game. Di bagian bawah, terdapat tombol btn_back dengan teks "Back to List" dan warna latar jingga terang (#FFA31A), berfungsi untuk kembali ke halaman utama.

Item_game.xml:

Layout ini menggunakan CardView sebagai komponen utama, yang memberikan tampilan dengan sudut membulat (`android:radius="16dp"`) dan bayangan (`elevation="6dp"`), menciptakan efek mengambang yang estetik dan modern. Warna latar belakang abu-abu terang `#D2D2D2` membuat elemen-elemen dalam kartu terlihat bersih dan kontras dengan teks serta tombol. Di dalam CardView, terdapat LinearLayout vertikal yang berfungsi sebagai kontainer semua elemen visual game. Pertama adalah ImageView dengan ID `imageGame`, yang menampilkan gambar game dengan tinggi tetap `180dp` dan `scaleType centerCrop`, memastikan gambar memenuhi ruang secara proporsional.

Setelah itu, TextView `textTitle` digunakan untuk menampilkan judul game dengan ukuran font besar dan tebal agar menarik perhatian. Di bawahnya, `textDesc` menampilkan ringkasan atau sinopsis singkat game, dibatasi hingga 2 baris dengan `ellipsize="end"` yang akan memotong teks dengan tiga titik jika terlalu panjang. Pada bagian bawah, terdapat LinearLayout horizontal yang menampung dua tombol: `buttonPage` untuk membuka halaman web game, dan `buttonDetail` untuk membuka detail lebih lengkap dalam DetailActivity. Keduanya diberi `backgroundTint` oranye (`#FFA31A`) dan warna teks gelap agar kontras, serta tata letak sejajar ke kanan (`gravity="end"`).

MainActivity.kt Compose:

MainActivity menginisialisasi GameViewModel menggunakan GameViewModelFactory, dan memantau state `navigateToUrl` untuk membuka browser jika user menekan tombol link. UI aplikasi dibangun melalui fungsi `setContent` yang menjalankan composable `GameApp`, dan dibungkus dengan tema `MODUL3COMPOSETheme`. Fungsi `GameApp` menampilkan dua layar tergantung pada state `selectedGameId`: jika null, akan menampilkan daftar game (`ListScreen`) dengan tombol detail dan link yang berinteraksi melalui ViewModel. Jika ada game yang dipilih, akan ditampilkan layar detail (`DetailScreen`) dari game tersebut. Aliran data bersifat satu arah (`unidirectional`), seluruh interaksi pengguna diproses di ViewModel, dan tampilan akan otomatis merespons perubahan state berkat penggunaan `collectAsState()`.

Game.kt Compose:

Di dalamnya terdapat *data class* bernama Game yang merepresentasikan struktur informasi sebuah game. Setiap objek Game menyimpan enam properti: id (identitas unik game dalam bentuk Int), titleRes (ID resource string untuk judul game), year (tahun rilis dalam bentuk String), plotRes (ID resource string untuk deskripsi cerita atau plot), imageRes (ID resource drawable untuk gambar game), dan Url (tautan ke halaman resmi atau toko game).

Selain itu, file ini juga mendeklarasikan sebuah List bernama gameList, yang berisi lima objek Game. Setiap game memiliki nilai properti yang diambil dari R.string dan R.drawable, yang menunjukkan bahwa teks dan gambar disimpan di resource file agar mendukung i18n (*internationalization*) dan efisiensi penyimpanan. Daftar ini menjadi sumber data utama yang digunakan oleh layar daftar game (ListScreen) dan detail game (DetailScreen) dalam aplikasi Compose.

DetailScreen.kt Compose:

Fungsi ini menerima objek Game dan lambda onBackPressed sebagai parameter untuk menangani aksi kembali. Di dalamnya digunakan Scaffold dengan TopAppBar berwarna kuning (yellow1) dan ikon panah kembali. Seluruh isi layar disusun dalam Column yang dapat discroll secara vertikal dengan konten terpusat secara horizontal.

Detail game ditampilkan mulai dari judul (dalam format vertikal dan kapital), tahun rilis, gambar (dengan Image dan ContentScale.Crop), serta deskripsi alur cerita (plot). Terdapat juga tombol Back to List di bagian bawah untuk kembali ke halaman sebelumnya. Kode ini memanfaatkan tema warna dan bentuk yang sudah ditentukan (Black1, softgrey1, yellow1) serta log Timber untuk mencatat navigasi.

ListScreen.kt Compose:

ListScreen menampilkan lima game favorit dalam bentuk daftar menggunakan LazyColumn, dengan setiap item direpresentasikan oleh fungsi GameItem. Setiap game ditampilkan dalam sebuah kartu (Card) yang menampilkan gambar, judul, dan plot singkat. Dua tombol disediakan: “Game Page” untuk membuka tautan eksternal, dan “Detail” untuk menavigasi ke halaman detail game. Tampilan ini juga memanfaatkan tema warna kustom seperti yellow1, softgrey1, dan Black1 untuk menjaga konsistensi desain UI. Selain itu, TopAppBar di bagian atas layar menampilkan judul aplikasi.

GameViewModel.kt Compose:

Kode ini adalah implementasi GameViewModel dan GameViewModelFactory dalam arsitektur MVVM menggunakan Kotlin dan Jetpack Compose. GameViewModel bertugas mengelola data game dan interaksi pengguna secara reaktif dengan memanfaatkan StateFlow. Ada tiga state utama yang dikelola: daftar game (games), game yang dipilih (selectedGameId), dan navigasi ke URL eksternal (navigateToUrl). Data awal game diinisialisasi dari gameList pada blok init, dan semua perubahan state dicatat menggunakan Timber untuk logging debug. Fungsi onGameSelected dan clearSelection mengatur pemilihan game untuk menampilkan detailnya, sedangkan onLinkClicked dan onUrlNavigated mengatur navigasi ke URL melalui intent eksplisit. GameViewModelFactory adalah kelas pembantu yang digunakan untuk membuat instance GameViewModel dengan parameter username, memastikan bahwa ViewModel dapat dibuat dengan dependensi yang diperlukan secara benar ketika dipanggil oleh sistem Android.

Step Into, Step Over, dan Step Out:

Debugger adalah alat penting dalam pengembangan perangkat lunak yang digunakan untuk menemukan dan memperbaiki kesalahan (bug) dalam kode program. Fungsi utama debugger adalah memungkinkan developer menjalankan program secara terkontrol, memeriksa nilai variabel, melacak alur eksekusi, dan memahami mengapa suatu bagian kode tidak berjalan sesuai harapan. Dengan debugger, programmer dapat menghentikan (pause) program pada titik tertentu yang disebut breakpoint, kemudian menganalisis kondisi program pada saat itu, sehingga memudahkan proses identifikasi kesalahan secara tepat dan efisien.

Cara menggunakan debugger biasanya dimulai dengan menetapkan breakpoint pada baris kode yang ingin diperiksa. Setelah breakpoint diaktifkan, program dijalankan dalam mode debugging. Ketika program mencapai breakpoint, eksekusi akan berhenti sementara, memungkinkan developer untuk melihat nilai variabel, kondisi memori, dan jalur logika yang sedang berjalan. Pada titik ini, debugger menyediakan kontrol eksekusi langkah demi langkah agar pengguna dapat mengeksplorasi jalannya program secara detail.

Fitur-fitur seperti Step Into, Step Over, dan Step Out membantu dalam navigasi selama debugging. **Step Into** digunakan untuk masuk ke dalam fungsi atau metode yang dipanggil di baris kode saat ini, memungkinkan developer melihat eksekusi kode di dalam fungsi tersebut secara rinci. **Step Over** menjalankan baris kode saat ini secara keseluruhan tanpa masuk ke fungsi yang dipanggil, cocok digunakan jika fungsi tersebut sudah dipastikan benar dan tidak perlu diperiksa lagi. Sementara itu, **Step Out** digunakan untuk keluar dari fungsi saat ini dan kembali ke kode pemanggil, berguna jika developer ingin mengakhiri penelusuran di dalam fungsi dan kembali ke konteks yang lebih luas. Ketiga fitur ini sangat membantu untuk memahami jalur eksekusi dan perilaku program secara mendalam, sehingga mempermudah proses debugging.

SOAL 2

Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya

Application Class dalam pengembangan aplikasi Android merupakan kelas fundamental yang berperan sebagai titik awal global sebelum komponen lain seperti Activity atau Service dijalankan. Kelas ini, yang merupakan turunan dari `android.app.Application`, berfungsi sebagai wadah untuk inisialisasi komponen-komponen penting yang dibutuhkan seluruh aplikasi, seperti library logging (Timber), database (Room), atau framework dependency injection (Dagger Hilt/Koin). Selain itu, Application Class juga menjadi tempat ideal untuk menyimpan data global yang perlu diakses berbagai bagian aplikasi, misalnya konfigurasi API atau informasi sesi pengguna. Dengan lifecycle yang mencakup seluruh durasi aplikasi, kelas ini memungkinkan developer memantau status aplikasi secara menyeluruh, seperti saat aplikasi berpindah antara latar belakang dan latar depan.

TAUTAN GIT

Berikut adalah tautan untuk source code yang telah dibuat.

<https://github.com/MuhammadErzaRaihan>